2025 年度 科学技術計算 第 2 回(通常) レポート

学籍番号 36714121

堀 航輔

2025 年 10 月 17 日

1 はじめに

このレポートでは、計算誤差に関するプログラムとその問題について述べる。なお、この課題において利用した Python のバージョンは全て 3.11.9 である。

2 演習課題

2.1 課題 1

ユーザーの入力した整数の累乗根を求めるプログラムを作成する。整数 1 からユーザーの入力した値までをループして求める手段もあるようだが、今回は”math.pow”関数と”is_integer”判定を使って実装した。

プログラム名は”kadai1.py”である。実行後、適当な正の整数を入力すると、2 から 7 までの指数についての累乗根が求められ、それが整数ならばその数を表示し、そうでないのなら”does not exist”を表示する。

実行結果の例(16 を入力)

Enter an integer: 16

2th root of 16 is 4

3th root of 16 does not exist

4th root of 16 is 2

5th root of 16 does not exist

6th root of 16 does not exist

7th root of 16 does not exist

2.2 課題 2

提示された数字を丸めて 10 進 4 桁の浮動小数点表現で書き、その丸め誤差を求める。問題文より”$0.abcd \times 10^n$”のような形式で表した後、本来の値と丸めた値の差の絶対値を求めることで答えを求める。今回は手計算で解く。以下に答えを示す。

問題	39.13	-375.461	0.0006324612	-3843.87
丸め表現	0.3943×10^2	-0.3755×10^3	0.6325×10^{-3}	-0.3844×10^4
相対誤差	0	0.039	0.0000000388	0.13

2.3 課題 3

厳密な有効桁数を求めるプログラムを書き、それを用いて課題 2 の値について検証する。関数”sigdigit”を定義する際に、”-math.log10(abs(x - approx)/abs(x))”で表される有効桁数の厳密な定義を用いた。

プログラム名は”kadai3.py”である。実行後、課題 2 の結果が自動的に検証される。関数は有効桁数の厳密な定義を返す。なお、与えられた値が全く同じだった場合は、エラーを回避するために”inf”が返される。

実行結果

inf

3.9835002254871146

4.212202162130665

4.470825339062525

2.4 課題 4

２次方程式の解についての問題。先に(1)と(4)を解く。(1)について、桁落ちというのは絶対値がほぼ等しい 2 つの数の減算を行った結果、有効数字の桁数が大幅に減少する現象のことである。つまるところ、解の公式内の「b^2-4ac」の部分の値が b に近しくなると、x1 または x2 の値が桁落ちを起こしてしまう。まとめるなら、b>0 かつ 4ac の値が非常に小さいなら x2 で、b<0 かつ 4ac の値が非常に小さいなら x1 で、桁落ちが発生し得る。(4)について、x1 は解が負の大きな値で、解の誤差は小さい値を保つ。x2 は解が負の小さな値で、解の誤差は拡大していく。こうなる理由は、x2 側で絶対値がほぼ等しい 2 つの数の減算を実行しているからだ。x2 は 0 に収束していくが、計算誤差は浮動小数点数の限界で決まるのも原因だ。以下に表を示す。

b の値	400	4000	40000
x1 の計算誤差	0.0	0.0	0.0
x2 の計算誤差	9.909850717804e-13	1.745421585042e-10	1.723532649933e-08

(2)と(3)の話に移る。プログラム名は”kadai4.py”である。実行後、与えられた abc に対して解を求める。その後、求めた解を元の式に代入してその値を表示する。途中で実数解の判定や、x1 と x2 の入れ替え作業も行っている。

実行結果の例(a=1,b=3,c=2 のとき)

解: -2.0, -1.0

x1 のとき 0.0

x2 のとき 0.0

2.5 課題 5

関数の無限級数展開を基に自然対数の近似値を求める。問題で示された通り、初期値の 1 に $x^n/n!$を足していくことで自然対数の底の値に近づいていくようになっている。プログラムの説明後にグラフを示す。

プログラム名は"kadai5.py"である。実行すると、定義された関数に連続で値を代入し、それを出力していく。ループ回数は 20 回とした。

実行結果

2.00000000000000000000e+00 0.264241

2.50000000000000000000e+00 0.080301

2.66666666666666651864e+00 0.018988

2.70833333333333303727e+00 0.003660

2.71666666666666634100e+00 0.000594

2.71805555555555544700e+00 0.000083

2.71825396825396836675e+00 0.000010

2.71827876984127003723e+00 0.000001

2.71828152557319224769e+00 0.000000

2.71828180114638451315e+00 0.000000

2.71828182619849290091e+00 0.000000

2.71828182828616871092e+00 0.000000

2.71828182844675936281e+00 0.000000

2.71828182845823018710e+00 0.000000

2.71828182845899490871e+00 0.000000

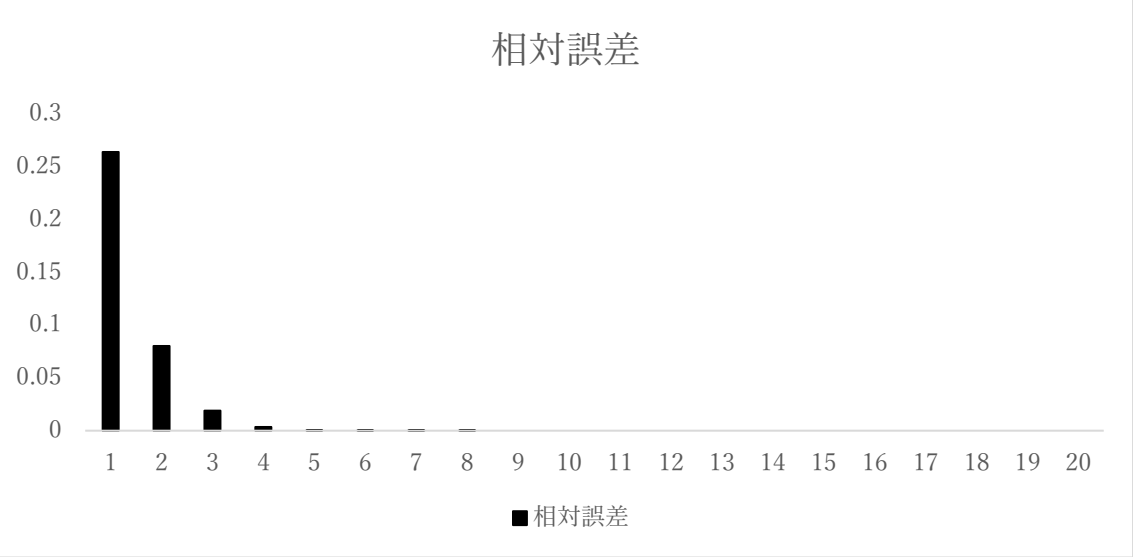
2.71828182845904287035e+00 0.000000

2.71828182845904553488e+00 0.000000

2.71828182845904553488e+00 0.000000

2.71828182845904553488e+00 0.000000

2.71828182845904553488e+00 0.000000



3 感想と考察

先に感想を述べる。コーディングの方は、Python 特有の書き方故に少し苦労したが、満足のいくものにできた。しかしレポートの方は、今まであまり書く機会が無かった為に、仕上げるるのに時間がかかった。総じて、課題を完成させるのに疲弊した。

次に今回の講義についての考察をする。講義を通して、計算機による数値計算というのは、「真の値」を扱う数学とは異なり、「誤差を伴う近似値」の操作であると分かった。特に、桁落ちや情報落ちといった誤差拡大のメカニズムは、アルゴリズムの選択や計算順序によって回避できることが示されている。例えば、桁落ちを避けるために式を変形したり、情報落ちを避けるために小さな値から順次加算したりと、計算の組み方や手法がいかに重要かを物語っている。この分野では精度の追求が必須であり、講義で学んだ数値表現の制約と誤差の発生、伝播メカニズムを常に意識し、これらを考慮して注意深く計算をする必要性を理解した。今後は、単にプログラムを組むだけでなく、数値表現の観点から計算手法の妥当性を検討することが重要になると考える。